

RedRhex 五階段課程式強化學習訓練與系統調整報告

執行摘要

本報告整理一套針對六足 wheg 機器人 RedRhex 的強化學習 (RL) 訓練與工程化改造流程，目標是在同一套 command-conditioned policy 下，同時具備直走 (forward)、側移 (lateral)、斜走 (diagonal) 與原地旋轉 (yaw turn) 等多技能 locomotion。原始做法採用單段式 multi-skill training，雖直接但容易產生技能間互相干擾與梯度混雜，導致「直走尚可、側移方向對但不夠積極、斜走薄弱、yaw 易 body contact 後死亡」等典型症狀。

本專案的核心改動可歸納為五個主軸：其一，將訓練流程由單段式改為五階段 curriculum (Stage1~5)，且以「同一 policy 連續微調、每段 checkpoint 串接」的方式降低技能汙染；並在 Stage5 (Mixed) 以整合潤化為目的，而非重學全部技能。其二，重構環境層控制邏輯為「技能模式感知」，並加入動作通道 gating (例如純側移時將 main_drive action 歸零、直走時鎖 ABAD) 以抑制 reward hacking 與不合理的通道濫用。其三，獎勵系統由「多項目互相打架」收斂為少數核心項並支援 ablation，同時搭配更完整的 TensorBoard logging；且修正 ABAD 相關變數初始化 bug、將 lateral 控制設計為兩階段 state machine (Preparation→Execution)。其四，加入 domain randomization 與 terrain curriculum (質量、摩擦、致動器強度、延遲、噪音、推擠等)，並設計「物理層 DR 嘗試 + device mismatch 時自動 fallback」以避免訓練中斷。其五，工程工具鏈全面補強：建立 train_stage_pipeline.sh 串接 stage、加入 GUI 預檢與健康檢查 gate、修正 play.py 可能載錯 TensorBoard event file 的風險、預設以 stop 起步降低起步不穩；並新增 eval_command_sweep.py 將「肉眼觀察」轉成可量化驗收 (速度誤差、角速度誤差、fall rate、roll/pitch RMS、pass/fail gate)。

在方法論層面，五階段 curriculum (屬於 curriculum learning) 常用於降低非凸訓練與多任務干擾，文獻指出以「由易到難」的課程式安排可提升泛化或收斂性。^① 本專案採用 PPO 作為基礎演算法，PPO 以 clipped surrogate objective 支援多次小步更新，在實務上常見於模擬機器人 locomotion。^② 針對 sim2real 的「reality gap」，以動力學/域隨機化 (dynamics/domain randomization) 提升策略對模型誤差的魯棒性亦為常見路線。^③

重要限制：使用者未提供「原始訓練 log、每次調整的實際時間戳、各版本 checkpoint 的完整數據與評測 CSV」，因此本報告凡涉及「具體時間、操作人、實際量測數值 (含 Hz、dB)」均標記為「未指定」；並在相應欄位提出可補齊的方法 (利用 logs/outputs 資料夾時間戳、git commit、TensorBoard event、eval CSV 等)。

研究背景與問題定義

RedRhex 為六足 wheg 機器人，訓練環境基於 Isaac Lab/Sim，並行 4096 個 environment，使用 PPO 演算法實作 RL 訓練。在結構設計上，ABAD (外展/內收) 提供額外控制通道，可改變足端落點與支撐多邊形，並在側移與轉向時影響 roll/yaw 耦合；因此若要讓 RL 真正「用上 ABAD」，模型與環境必須顯式保留 ABAD 對支撐幾何的影響，否則 ABAD 可能退化成噪音來源。

問題起點可分成三類：

第一類是訓練穩定性與可學性 (learnability) 問題：單段式 multi-skill 訓練使同一 actor 同時承受多種任務梯度，易導致不同技能的控制需求與 reward shaping 拉扯，最終產生保守但不完成任務的策略 (典型表現如 lateral 不積極、yaw 先倒地終止)。第二類是程式正確性：訓練過程曾出現 ABAD 相關變數未初始化造成的 UnboundLocalError。第三類是工具鏈誤判：播放/驗收時可能把 TensorBoard 的 event 檔誤當 checkpoint

載入，導致 UnpicklingError；以及起步若直接給 forward 命令可能放大不穩，使「工具/設定問題」被誤判為「模型壞掉」。

本報告採用的研究策略是：以 curriculum learning 拆解技能、以 action gating 與 reward simplification 把策略空間限制在「物理合理且可驗收」的子空間、以 domain randomization 提升泛化/降低 reality gap、以標準化的 command sweep 建立量化驗收，使訓練與比較具備可重現性與可溝通性。 ⁴

調整與實驗流程時間序列表

說明：下表依「邏輯因果順序」整理各次調整。由於未提供原始實驗紀錄，本表中「時間、操作人、實際量測（含 Hz、dB）」多為未指定。建議以 logs/rsl_rl/... 的 YYYY-MM-DD_HH-MM-SS 資料夾命名、Hydra outputs/ 的時間戳、以及 git commit history 補齊精確時間與版本對應。

步驟 ID	時間	操作人	設備/軟體	具體調整項目與數值/音高	預期目的	實際觀察果
S0	未指定	未指定	Isaac Lab/Sim + PPO；並行 4096 env	單段式 multi-skill（forward/lateral/diagonal/yaw 一次混訓）	一次學會多技能	直走尚可 lateral 有向但慢；diagonal 弱；yaw body contact 死亡
S1	未指定	未指定	redrhex_env.py	修正 ABAD 計算時機：提前初始化，避免 UnboundLocalError	解除 crash，讓訓練可持續	訓練不再變數未初始化中斷（節未指定）
S2	未指定	未指定	redrhex_env.py（lateral 控制）	lateral two-phase：Preparation（P 控制回初始）→ Execution（鎖 main_drive、ABAD 步進）；FSM：GO_TO_STAND→LATERAL_STEP	提升側移可學性、避免卡死	lateral 動更流暢（節未指定）
S3	未指定	未指定	redrhex_env.py + env_cfg.py	獎勵重構：減少懲罰、提升 velocity tracking、加入腿部動作獎勵、權重集中於 config、全項 logging、支援 ablation	降低 reward 互相衝突與 reward hacking；提升可調性	mean reward 負轉正；為更積極趨勢更可蹤（數值指定）
S4	未指定	未指定	redrhex_env.py	command mode 分類（FWD/LAT/DIAG/YAW/OTHER）；依 mode 分流 reward/gating/diagnostics	讓不同技能用不同邏輯，降低互相打架	能更清楚 isolate 問題（細節指定）

步驟 ID	時間	操作人	設備/軟體	具體調整項目與數值/音高	預期目的	實際觀察果
S5	未指定	未指定	redrhex_env.py / env_cfg.py	action gating：純 lateral 時 main_drive action=0；forward 時 (lock_abad_in_forward=True) ABAD action=0	強迫「用對通道做事」，抑制作弊	混訓時 lateral 不靠亂轉 main_dr 偷取速度 (未指定)
S6	未指定	未指定	env_cfg.py	forward gait prior：stance 65%/swing 35%；stance angle 60°/swing angle 300°；鼓勵 transition window ≥ 4 腳接觸	降低探索空間、提升起步與步態穩定	起步更穩、較少高頻搖 (未指定)
S7	未指定	未指定	訓練流程設計	由單段改五階段 curriculum：Stage1 forward-only；Stage2 lateral-only；Stage3 diagonal-only；Stage4 yaw-only；Stage5 mixed；同一 policy 連續微調	降低技能干擾，提升可診斷性	能逐技能斂後再整 (量化未定)
S8	未指定	未指定	train_stage_pipeline.sh	pipeline 一鍵跑完五階段；預設 full resume (policy+optimizer+iteration)；並提供 GUI 預檢 (precheck_envs=64、precheck_iters=120) 與正式訓練 (num_envs=4096；s1=8000,s2=8000,s3=9000,s4=10000,s5=12000)	提升可重現性、避免整晚白跑	可先確認「不是一生就觸地亡」再長練 (量化指定)
S9	未指定	未指定	train_stage_pipeline.sh	解析工具 fallback：rg 不存在改用 grep；避免 pipeline 因系統工具缺失中斷	降低環境相依性	pipeline 因 rg 缺失中斷 (未定)
S10	未指定	未指定	Hydra	Hydra override 統一：用 env.stage=1；新增 key 用 +env.xxx；避免 “Could not override 'stage' / Key not in struct”	保證指令可複製、可重現	覆寫不再敗 (未指定)
S11	未指定	未指定	env_cfg.py	物理與泛化：static_friction=1.2、dynamic_friction=1.0、restitution=0；DR 範圍：mass 0.90–1.10、friction 0.50–1.25、actuator 0.85–1.15、latency 0–2 steps、random push 等	提升抓地合理性與 sim2real 魯棒性	DR 能做物理層遇 device mismatch 自動 fallback 避免訓練掉
S12	未指定	未指定	play.py + eval_command_sweep.py	播放與驗收：play.py 只接受 model_*.pt、誤傳 event file 會 fallback；初始命令預設 stop；從 checkpoint 路徑自動推 env.stage；新增 eval_command_sweep 量化驗收與 CSV	降低誤判、把「看起來會動」轉成指標	可分離回問題 vs 訓練問題；收可輸出 PASS/FAIL (實際比未指定)

補充（版本對照）：會議紀錄指定舊版穩定節點 `ca4b1ab` 與新版 `eca15ea` (5stage-new-stable) 作比較基準；但未提供兩者的實際日期/metrics，應透過 git log 補齊。

調整項目逐一解析

本節以「原因 → 方法 → 參數變動 → 測量方法 → 結果與量化分析（未指定則提出設計） → 誤差來源與改進」的格式，逐項對應上表 S0-S12。

從單段式到五階段 curriculum 的因果鏈

原因：單段式 multi-skill 訓練的梯度來源混雜，reward/控制需求互相拉扯，特別在 lateral 與 yaw 上容易拖垮整體穩定性；同時也難以定位「哪一項技能是瓶頸」。

方法：採五階段 curriculum：Stage1 直走打底；Stage2 單獨拉起側移；Stage3 將 vx+vy 融合成 diagonal；Stage4 單獨處理 yaw（允許 per-leg signed main-drive 以提升轉向可行性）；Stage5 再做全技能 mixed 整合並潤化。

參數變動（已知）：提供 pipeline 參考配置：預檢（precheck_envs=64、precheck_iters=120、num_envs=512）；正式訓練 num_envs=4096；stage iteration 上限 s1=8000、s2=8000、s3=9000、s4=10000、s5=12000。

測量方法（已知）：TensorBoard 分階段監控指標：mean_episode_length、mean_reward、terminated、rew_fall、diag_base_height；以及 stage-specific diagnostics（如 Stage2 的 diag_lateral_fsm_state、Stage4 的 diag_roll_rms/diag_pitch_rms）。

結果分析（未指定，給出可直接套用的量化框架）：

- 1) 以「每階段收斂速度」比較：到達 mean_episode_length 穩定平台所需 iteration；
- 2) 以「技能保留」比較：Stage5 的 forward 相關指標是否維持 Stage1 水準（以 diag_cmd_vx、rew_forward_gait、diag_forward_duty_ema 等 proxy），避免 catastrophic forgetting；
- 3) 以「跨技能平均表現」比較：stage5 command sweep 中 forward/lateral/diagonal/yaw 的 pass ratio 是否同時達標（見後述 acceptance gate）。

可能誤差來源：episode 終止條件（terminations）與 reward shaping 的耦合可能造成「表面穩定但不追蹤命令」的假收斂；需搭配 command sweep 才能排除。

全段接續學習的工程化：full resume、GUI 預檢、health gate

原因：若每個 stage 只載入 policy 而不延續 optimizer/runner 狀態，表面上像接續訓練但延續性弱，易造成前階段技能遺忘；另一方面，headless 長訓練若一開始就「出生即死」，會浪費整晚算力。

方法與參數：

- full resume：train_stage_pipeline.sh 預設 RESUME_POLICY_ONLY=0，延續權重、optimizer state 與 iteration/runner 狀態。
- GUI 預檢：加入 --precheck_gui、--precheck_stage 等參數，先小規模跑 2-5 分鐘，確認不是一出生就 terminated。
- health gate：從 log 抓 Mean episode length、Episode_Termination/terminated 等指標；若不合理就中止 pipeline，避免錯誤放大。
- 解析工具 fallback：rg 缺失時改用 grep，避免 pipeline 因系統工具差異中斷。

結果（依文件敘述）：上述設計的直接效果是把「課程式訓練」從概念落地成可整夜自動跑、且具基本保護機制的工程流程；並降低「工具鏈問題」被誤判為「模型壞」的概率。

改進建議：health gate 的閾值本身應納入版本控制（config 化）並與 command sweep 結果連動；否則可能出現「train 看起來健康、但技能其實沒追蹤」的漏網之魚。

獎勵系統重構與 reward hacking 風險管理

原因：原始獎勵函數過於複雜且懲罰過多，容易導致學習困難與 reward hacking，使策略採取保守或鑽漏洞行為；另有 ABAD 變數計算時機錯誤導致訓練中斷。這類「目標錯置/規格投機（specification gaming）」在安全研究中被系統性討論。⁶

方法：

- Bug 修正：ABAD 計算提前，確保變數初始化。
- 行為 shaping：lateral 兩階段 movement（Preparation→Execution）避免卡死並強迫「先站穩再側移」。
- 獎勵簡化與 ablation：cfg 開啟 use_simplified_rewards，提供 ablation_flags；env 端依 _is_reward_enabled() 決定是否啟用 reward，以支援系統化「哪個 reward 有用」的分析。
- 命令感知 reward：包含 axis_suppression（未命令軸速度抑制）與 mode_specialization（側移/旋轉/斜向各自 shaping），搭配 forward gait prior。
- 姿態誤判修正：以 projected_gravity 與 reference_projected_gravity 的內積計算 tilt_angle，避免座標系微偏使 roll/pitch 判斷失真，導致策略過度保守。

結果（文件已記載但數值未提供）：TensorBoard 顯示 mean reward 由負轉正、行為更積極、各 reward 趨勢更可追蹤。

建議的量化比較（未指定資料下的可交付格式）：

- 指標選擇：AUC(mean_reward)、terminated 比例、fall rate、速度追蹤誤差 RMS；
- 比較設計：舊 reward vs 新 reward，至少 3 個不同 seed；以 bootstrap 估計 95% CI（避免把單次跑的偶然性當成結論）；
- ablation：按 ablation_flags 逐項關閉 axis_suppression、mode_specialization、gait prior 等，觀察 pass ratio 與失敗型態是否改變。

控制通道分工：致動器分群、action gating、lateral state machine

原因：若不約束動作通道，策略可能用「不該用」的關節完成任務（例如用 main_drive 亂轉偷出側向速度），造成 reward hacking；同時也會使多技能訓練更不穩、更難 debug。

方法與參數（已知）：

- 物理材質：static_friction=1.2、dynamic_friction=1.0、restitution=0，以 multiply 合併模式，避免過滑或過彈導致策略學到抖動或跳躍取巧。
- 致動器分群：main_drive（速度控制、damping 大）、ABAD（位置控制、stiffness/damping 中等）、damper（高 stiffness 鎖定、不讓 RL 控制）。
- gating：lock_abad_in_forward=True、lock_main_drive_in_lateral=True、require_stand_before_lateral=True（含 tolerance）。
- action gating 實作：純 lateral 時 main_drive action=0；forward 且 lock_abad_in_forward=True 時 ABAD action=0。
- lateral FSM：GO_TO_STAND→LATERAL_STEP，確保側移前姿態與接觸條件合理。

結果與量化（未指定，對應可觀測量）：

- 若 gating 有效，應看到：

- (1) lateral 命令下 main_drive 的 action/histogram 顯著縮小；
- (2) vy 追蹤誤差下降、diag_lateral_vel 絕對值上升且方向正確；
- (3) 接觸分佈更穩（contact histogram 不呈現異常偏態）。

播放與驗收工具鏈：從「看起來會動」到「可出分數」

原因：肉眼觀察容易只看到直走還行就誤以為整體可用；同時工具鏈若載錯檔案或起步命令過激，會把播放問題誤判為訓練問題。

方法：

- eval_command_sweep：固定命令集合逐一測試，統計速度誤差、角速度誤差、fall rate、roll/pitch RMS、接觸統計，並用 acceptance gate 判定 PASS/FAIL。
- stage5 驗收門檻（已知）：warmup_steps=120、sweep_steps=600、accept_duration_s=2.0；accept_vx_abs=0.15、accept_vy_abs=0.15、accept_wz_abs=0.40；accept_max_fall_rate=0.20；accept_overall_pass_ratio=0.70 等。
- play.py 防呆：只接受 model_*.pt；誤傳 events.out.tfevents... 時自動 fallback；CLI 預設起步改 stop；自 checkpoint 路徑推斷 env.stage，避免拿 Stage4 模型卻用 Stage1 設定播放。
- 鍵盤控制與 external_control：external_control=True 時環境跳過自動重採樣命令，讓外部（鍵盤/評測）完全掌控。

與官方/原始來源的對應：Isaac Lab 文件提供將策略匯出為 TorchScript (JIT) 與 ONNX 的函式介面（export_policy_as_jit / export_policy_as_onnx），可支持部署與展示流程標準化。⁷

結果（部分已在會議紀錄以驗收輸出形式呈現）：會議紀錄示例中出現「PASS: forward, diag_left / FAIL: left, right, ...」與 stability.fall_rate、roll_rms 等數值，顯示該工具可輸出可溝通的量化摘要；但完整樣本量與時間窗未指定，不能視為最終統計結論。

Domain randomization 與 sim2real：讓策略不怕「世界不是模擬器」

原因：僅在固定模擬參數下訓練，策略可能對 simulator 的特定摩擦、質量、延遲過擬合，導致 sim2real 時性能崩潰。文獻顯示透過（影像或動力學）domain randomization 可降低 reality gap，並在一定條件下促進 transfer。⁸

方法與參數（已知）：cfg 定義 DR 範圍：mass 0.90–1.10、friction 0.50–1.25、actuator strength 0.85–1.15、observation latency 0–2 steps、observation noise、random push；並設計 terrain curriculum 隨 stage 提升難度。

工程可靠性：嘗試透過 root_physx_view 直接設定質量與材質；若遇 PhysX tensor device 與 PyTorch device 不一致，會自動關閉 physical DR 並 warning，以避免訓練炸掉。

量化建議（未指定資料下的可交付方式）：

- 在「同一 checkpoint」下，分別於 DR-on 與 DR-off 的多組隨機種子環境做 eval_command_sweep，對比 overall_pass_ratio 的平均與 CI；
- 若有真機，採「同命令集」對照速度追蹤誤差分佈（如 0.15 m/s 以內的比例）以建立 sim2real gap 指標。

量測與統計分析設計

本節提供一套「你就算現在手上只有 logs 與 checkpoint，也能補齊一份教授會買單的量化分析」的設計。所有未提供的原始數據欄位仍標為未指定。

指標定義與對應資料來源

訓練監控 (TensorBoard) 指標建議至少包含：Train/mean_reward、Train/mean_episode_length、Episode_Termination/terminated、Episode_Reward/rew_fall、Episode_Reward/diag_base_height 等；並依 stage 觀察特定 diagnostics (如 diag_lateral_fsm_state、diag_wz_error、diag_roll_rms、diag_pitch_rms)。

驗收 (eval_command_sweep) 指標建議至少包含：

- 速度追蹤誤差： $e_v = \|[v_x, v_y]_{\text{cmd}} - [v_x, v_y]_{\text{base}}\|$ (m/s)
- 角速度追蹤誤差： $e_\omega = |w_z^{\text{cmd}} - w_z^{\text{base}}|$ (rad/s)
- 穩定度：roll/pitch RMS、base height 偏移
- 失敗率：fall rate (摔倒次數/評測回合數)
- 通過率：skill pass ratio 與 overall pass ratio (依 acceptance thresholds)

資料存放位置與可追溯性：模型 checkpoint (model_.pt)、TensorBoard events、Hydra outputs (配置快照) 皆在專案既定路徑結構中；路徑同時提供時間戳，可回推「何時跑了哪次訓練」。

缺失資料補齊清單

缺失項	為何重要	建議補齊方法	產出格式
各次調整的精確時間戳	讓「時間序列」可驗證	以 logs/rsl_rl/*/YYYY-MM-DD_HH-MM-SS 與 outputs/YYYY-MM-DD/HH-MM-SS 自動擷取	表格 (ISO 8601)
操作人與版本	可追責、可重現	git commit hash + commit message + Branch 名稱 (例：ca4b1ab vs eca15ea)	表格 (hash/branch)
每次調整前後的定量指標	使「改了有用」可證明	每版 checkpoint 跑一次 eval_command_sweep 並輸出 CSV；同時匯出 TensorBoard scalars	CSV + 圖表
隨機種子與重複次數	避免單次結果偶然	每個設定至少 3 seeds；報告平均±標準差/CI	統計摘要
若教授堅持要 Hz、dB (聲學量)	與本題本質不直接相關，但可加分	在模擬/真機錄音：麥克風固定距離、固定採樣率；以 STFT 得主頻與 SPL(dB)	附錄：頻譜圖 (未指定)

小提醒 (帶一點善意的吐槽)：本專案核心是 locomotion policy 的控制與穩定，不是聲學實驗；若表格一定要填 Hz 與 dB，建議把 Hz 解讀為「控制迴圈/步態頻率」，把 dB 解讀為「聲壓級或能耗 proxy 的對數尺度」，並在報告中明確註記定義，免得教授以為你把 whet 當音叉在敲。

統計比較建議

- **主比較問題**：新版 (五階段 + gating + reward simplification + DR) 是否在 stage5 命令集合上提高 overall_pass_ratio，並維持 forward 的穩定性。
- **統計量**：對每個 seed 的 overall_pass_ratio 計算平均與 95% bootstrap CI；對 fall rate 以二項分佈比例 CI 報告；對誤差 RMS 採 bootstrap 或 t 分佈近似 (註記假設)。
- **控制變因**：PPO 超參數保持一致 (若未指定則需從 Hydra outputs 補齊)，只改環境與課程設定，避免把優化器差異誤判為環境改動效果。

口頭報告要點

- 研究目標：同一個 command-conditioned policy 同時具備 forward/lateral/diagonal/yaw 多技能 locomotion，並能被量化驗收。
- 原始瓶頸：單段式 multi-skill 訓練造成技能互相干擾與難診斷，尤其 lateral 與 yaw 容易拖垮整體。
- 方法總覽：五階段 curriculum (Stage1→5)，同一 policy 串接 checkpoint 連續微調。
- 工程關鍵：train_stage_pipeline.sh 一鍵串接 + full resume (含 optimizer/iteration) + GUI 預檢 + health gate，避免整晚白跑。
- 控制關鍵：command mode 分流 + action gating (forward 鎖 ABAD、lateral 鎖 main_drive)，把「分工」寫死在環境裡抑制作弊。
- 步態先驗：forward gait prior (stance 65%/swing 35% 等) 降低探索空間，提升起步與步態穩定。
- 獎勵重構：由多懲罰改為少數核心項 + logging + ablation，可系統化回答「哪個 reward 有用」。
- 泛化工程：DR (質量/摩擦/延遲/噪音/推擠) + terrain curriculum；並加入物理層 DR 的自動 fallback。
- 驗收工具：eval_command_sweep，把「看起來有動」轉成速度誤差、fall rate、roll/pitch RMS、PASS/FAIL gate，且可輸出 CSV。
- 工具鏈防呆：play.py 防止載錯 events 檔、預設 stop 起步、可自動推斷 env.stage，降低誤判模型壞掉。
- 目前限制：缺少原始 log/CSV/時間戳，報告中的數值多為未指定；但已提出可補齊路徑 (logs/outputs/git/eval CSV)。
- 文獻定位：PPO (Schulman et al.) 是 locomotion 常用 policy gradient；curriculum learning 與 domain/dynamics randomization 為降低難度與 reality gap 的主流方法。 9

常見問答

問：為什麼一定要五階段？四階段或混訓不行嗎？

答：五階段的設計核心是把 diagonal 與 yaw 也「顯式當成技能」單獨收斂；若只靠 forward/lateral 外推，斜走通常不夠穩也不夠強，而 yaw 又最容易先把整體拖垮，因此需要隔離訓練再整合。

問：action gating 會不會把 RL 綁死，反而學不到更好的策略？

答：gating 的目的是排除「物理上不合理或明顯作弊」的通道使用（例如純側移卻用 main_drive 亂轉偷側向速度），縮小搜尋空間提升可學性；最終仍可在 Stage5 (Mixed) 逐步鬆綁或以 ablation 檢驗其必要性。

問：你怎麼證明改動讓模型變好，而不是剛好跑到好 seed？

答：以 command sweep 的 pass ratio (含 CI) 做主要指標，至少 3 seeds；比較新版/舊版 checkpoint 在同一 eval_profile 下的 overall_pass_ratio 與 fall rate，並公開每次的 Hydra config snapshot 以確保可重現。

問：為什麼 full resume 這麼關鍵？只載入 policy 不行嗎？

答：只載入 policy 會讓 optimizer state 與 runner 狀態重置，形式上像「接著訓練」但延續性弱，容易讓 curriculum 的接續效果不足、甚至更易遺忘前一階段技能；full resume 才更接近真正的「接續學習」。

問：你們如何避免「forward 被後續 stage 洗掉」？

答：環境加入 forward 防遺忘機制 (forward 模式殘差上限、stage-specific residual scale、Stage5 保留 forward reward multiplier)；此外以 stage5 指標檢查 forward 是否仍有反應。實際效果需用 stage5 sweep 中 forward 子集合的 pass ratio 或追蹤誤差分佈量化。

問：你們的驗收門檻 (例如 accept_vx_abs=0.15) 是怎麼訂的？

答：目前門檻已在 eval_command_sweep 指令中明確列出，屬「工程化 acceptance test」；理想做法是以真機能力或任務需求反推 (例如希望 ± 0.15 m/s 內維持 2 秒)，並用敏感度分析 (門檻上下浮動) 檢查結論是否穩健。

問：DR 為什麼要做？會不會讓訓練變慢、變不穩？

答：DR 是為了降低 reality gap；確實可能增加訓練難度，因此採「範圍可控 + terrain curriculum 隨 stage 加大」設計。若物理層 DR 因 device mismatch 失敗，系統會自動 fallback，避免訓練中斷。 ¹⁰

問：為什麼要鍵盤控制與 external_control？

答：因為 RL debug 常見困境是「reward 看起來很好，但它到底學到什麼？」鍵盤控制可直接下達固定命令觀察策略反應；external_control=True 時環境不再自行重採樣命令，避免干擾展示與驗收。

問：你如何避免播放時把 TensorBoard event file 當 checkpoint？

答：play.py / eval 都加入防呆：只接受 model_.pt；若誤傳 events.out.tfevents... 則自動 fallback 到同資料夾最近的 model_.pt，並建議起步用 stop 以降低起步不穩造成的誤判。

問：教授如果問「PPO/課程式學習/域隨機化」是否有文獻支持？

答：PPO 由 Schulman 等提出，以 clipped surrogate objective 支援穩定的 policy gradient 更新，常用於機器人 locomotion；curriculum learning 由 Bengio 等提出，強調由易到難可能改善訓練與泛化；domain/dynamics randomization 被 Tobin、Peng 等用於降低 reality gap 與 sim2real transfer。 ¹¹

```
flowchart TD
  A[CAD/URDF 匯入與 USD 驗證] --> B[IsaacLab 環境層\nnredrhex_env.py + env_cfg.py]
  B --> C[Stage1 Forward-only]
  C --> D[Stage2 Lateral-only\nnresume stage1 checkpoint]
  D --> E[Stage3 Diagonal-only\nnresume stage2 checkpoint]
  E --> F[Stage4 Yaw-only\nnresume stage3 checkpoint]
  F --> G[Stage5 Mixed integration\nnresume stage4 checkpoint]
  G --> H[TensorBoard 監控\nreward/diagnostics]
  G --> I[eval_command_sweep\nPASS/FAIL + CSV]
  I --> J[play.py 展示\nkeyboard + stop 起步]
  J --> K[Sim2Real / 真機對照測試]
```

（上圖對應五階段設計、pipeline 串接、TensorBoard 監控、command sweep 驗收與播放展示的整體流程。）

¹ ⁴ <https://dl.acm.org/doi/10.1145/1553374.1553380>

<https://dl.acm.org/doi/10.1145/1553374.1553380>

² ⁹ ¹¹ Proximal Policy Optimization Algorithms

https://arxiv.org/abs/1707.06347?utm_source=chatgpt.com

³ ¹⁰ <https://arxiv.org/abs/1710.06537>

<https://arxiv.org/abs/1710.06537>

⁵ Basic Override syntax

https://hydra.cc/docs/advanced/override_grammar/basic/?utm_source=chatgpt.com

⁶ <https://arxiv.org/pdf/1606.06565>

<https://arxiv.org/pdf/1606.06565>

⁷ isaacsim_rl — Isaac Lab Documentation

https://isaac-sim.github.io/IsaacLab/main/source/api/lab_rl/isaacsim_rl.html?utm_source=chatgpt.com

8 Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World
https://arxiv.org/abs/1703.06907?utm_source=chatgpt.com